3Sum Closest

Two approaches, visualized execution, and the core logic to remember

Brute Force · O(n³) | Two Pointers · O(n²)

Reference notes · ThreeSumClosest.java

Problem Statement

Given an integer array `nums` and an integer `target`, find three integers at distinct indices whose sum is as close as possible to `target`, and return that sum. Assume exactly one solution.

Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	O(n³)	O(1)	Check every triplet, track smallest diff, early-exit on exact match
Two Pointers	O(n²)	O(1)	Fix one number, two-pointer the rest, always keep the best diff

Big picture: This is 3Sum's exact skeleton with the success condition swapped — instead of collecting every triplet that hits exactly zero, track a running "best sum so far" by absolute difference from `target`, and only short-circuit early on an exact match.

1 · Brute Force

O(n³) time **O(1) space**

Try every (i, j, k) → **diff = |sum - target|** → **smaller diff? update best** → **exact match? return early**

```java
// try everything, keep the best diff, stop early on exact match
private static int bruteForce(int[] a, int target) {
    if (a == null || a.length < 3) {
        throw new IllegalArgumentException("Array must contain at least 3 elements");
    }
    int closestSum = a[0] + a[1] + a[2];
    int minDiff = Math.abs(closestSum - target);

    for (int i = 0; i < a.length - 2; i++) {
        for (int j = i + 1; j < a.length - 1; j++) {
            for (int k = j + 1; k < a.length; k++) {
                int sum = a[i] + a[j] + a[k];
                int diff = Math.abs(sum - target);
                if (diff < minDiff) {
                    minDiff = diff;
                    closestSum = sum;
                }
                if (sum == target) {
                    return sum;
                }
            }
        }
    }
    return closestSum;
}
```

Recall trick: "Try everything, keep the best diff, stop early on exact match." Straightforward and correct, but O(n³) — too slow for larger inputs.

2 · Two Pointers

O(n²) time **O(1) space**

Sort array, fix a[i] → **left = i+1 right = n-1** → **diff < minDiff? update best sum** → **exact match? return sum** → **too small→left++ too big→right--**

```java
// fix one number, two-pointer the rest, always keep the best diff
private static int usingTwoPointers(int[] a, int target) {
    Arrays.sort(a);
    int resultSum = a[0] + a[1] + a[2];
    int minDiff = Integer.MAX_VALUE;

    for (int i = 0; i < a.length - 2; i++) {
        int left = i + 1;
        int right = a.length - 1;

        while (left < right) {
            int sum = a[i] + a[left] + a[right];
            int diffToTarget = Math.abs(target - sum);

            if (diffToTarget < minDiff) {
                resultSum = sum;
                minDiff = diffToTarget;
            }

            if (sum == target) {
                return sum;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
    }
    return resultSum;
}
```

Trace for `a = [-1,0,1,2,-1,-4]` → sorted `[-4,-1,-1,0,1,2]`, `target = 4`:

i	a[i]	left/right walk	best sum so far
0	-4	sums -3, -3, -2, -1 as left advances (all < target)	-1 (diff 5)
1	-1	sums 0, 1, 2 as left advances (all < target)	2 (diff 2)
2	-1	sums 1, 2 — no improvement, diff ties at 2	2 (diff 2)
3	**0**	**sum 3 (diff 1) → best so far**	**3 (diff 1)**

Recall trick: "Fix one number, two-pointer the rest, always keep the best diff." Same skeleton as 3Sum, but instead of collecting exact-zero matches, every sum along the way is a candidate — just keep whichever is closest to `target`. Result here is 3 — the maximum possible sum (0+1+2) turns out to be the closest this array can get to `target = 4`. The method now sorts internally, so it's self-contained and safe to call with any unsorted array.

TL;DR — For Future Me

What to remember

- **Best solution:** Two pointers, O(n²) — "fix one number, two-pointer the rest, always keep the best diff instead of only exact matches."
- **Brute force** is O(n³) — correct but only fine for small inputs.
- **Underlying pattern:** this is 3Sum's skeleton with the success condition swapped — instead of collecting all triplets that hit exactly zero, track a running "best so far" and only short-circuit on an exact match.
- **Now self-contained:** `Arrays.sort(a)` was moved inside `usingTwoPointers`, so the method no longer relies on the caller pre-sorting the array in `main()`.